

GitHub-Ready locationController.js

Cleaned for GitHub upload. The controller contains location CRUD operations for favorite locations and does not contain hard-coded credentials.

```
const Location = require('../models/Location');

// @desc    Add a favorite location
// @route    POST /api/locations
// @access   Private
const addLocation = async (req, res) => {
  try {
    const { city, country } = req.body;

    if (!city || !country) {
      return res.status(400).json({ success: false, message: 'Please provide both city and country' });
    }

    // Check if user already favorited this location
    const alreadyFavorited = await Location.findOne({
      user: req.user._id,
      city: { $regex: new RegExp(`^${city.trim()}$`, 'i') },
      country: { $regex: new RegExp(`^${country.trim()}$`, 'i') }
    });

    if (alreadyFavorited) {
      return res.status(400).json({ success: false, message: 'Location is already in your favorites' });
    }

    const location = await Location.create({
      city: city.trim(),
      country: country.trim(),
      user: req.user._id
    });

    return res.status(201).json({
      success: true,
      data: location
    });
  } catch (error) {
    console.error('Error adding location:', error.message);
    return res.status(500).json({ success: false, message: 'Server error while adding location' });
  }
};

// @desc    Get all favorite locations for a user
// @route    GET /api/locations
// @access   Private
const getLocations = async (req, res) => {
  try {
    const locations = await Location.find({ user: req.user._id }).sort({ createdAt: -1 });

    return res.json({
      success: true,
      count: locations.length,
      data: locations
    });
  } catch (error) {
    console.error('Error fetching locations:', error.message);
    return res.status(500).json({ success: false, message: 'Server error while fetching locations' });
  }
};

// @desc    Update a favorite location
// @route    PUT /api/locations/:id
// @access   Private
const updateLocation = async (req, res) => {
  try {
    const { city, country } = req.body;
    const locationId = req.params.id;

    if (!city || !country) {
      return res.status(400).json({ success: false, message: 'Please provide both city and country' });
    }

    // Find location and check authorization
```

```

const location = await Location.findById(locationId);

if (!location) {
  return res.status(404).json({ success: false, message: 'Location not found' });
}

// Make sure user owns location
if (location.user.toString() !== req.user._id.toString()) {
  return res.status(401).json({ success: false, message: 'User not authorized to update this location' });
}

// Check if the update creates a duplicate of another favorite location
const duplicate = await Location.findOne({
  user: req.user._id,
  _id: { $ne: locationId },
  city: { $regex: new RegExp(`^${city.trim()}$`, 'i') },
  country: { $regex: new RegExp(`^${country.trim()}$`, 'i') }
});

if (duplicate) {
  return res.status(400).json({ success: false, message: 'This location is already in your favorites' });
}

location.city = city.trim();
location.country = country.trim();
const updatedLocation = await location.save();

return res.json({
  success: true,
  data: updatedLocation
});
} catch (error) {
  console.error('Error updating location:', error.message);
  return res.status(500).json({ success: false, message: 'Server error while updating location' });
}
};

// @desc    Delete a favorite location
// @route    DELETE /api/locations/:id
// @access   Private
const deleteLocation = async (req, res) => {
  try {
    const locationId = req.params.id;

    // Find location and check authorization
    const location = await Location.findById(locationId);

    if (!location) {
      return res.status(404).json({ success: false, message: 'Location not found' });
    }

    // Make sure user owns location
    if (location.user.toString() !== req.user._id.toString()) {
      return res.status(401).json({ success: false, message: 'User not authorized to delete this location' });
    }

    await Location.deleteOne({ _id: locationId });

    return res.json({
      success: true,
      message: 'Location removed from favorites',
      data: {}
    });
  } catch (error) {
    console.error('Error deleting location:', error.message);
    return res.status(500).json({ success: false, message: 'Server error while deleting location' });
  }
};

module.exports = {
  addLocation,
  getLocations,
  updateLocation,
  deleteLocation
};

```

GitHub upload: save this as **locationController.js** in your controllers folder. Keep secrets and environment files such as **.env** out of the repository.